

SHARIF
UNIVERSITY OF TECHNOLOGY



Signals & Systems

Computer Assignment 5

Aria Fateh Kerdari
403110382

July 19, 2026

1 Ideal Sampling and Sinc Reconstruction

1.1 Sampling

The Nyquist sampling rate is $F_n = 180$ Hz. The function is depicted in Figure 1. Then we sampled the mentioned signal at different rate and the result is shown in Figure 2.

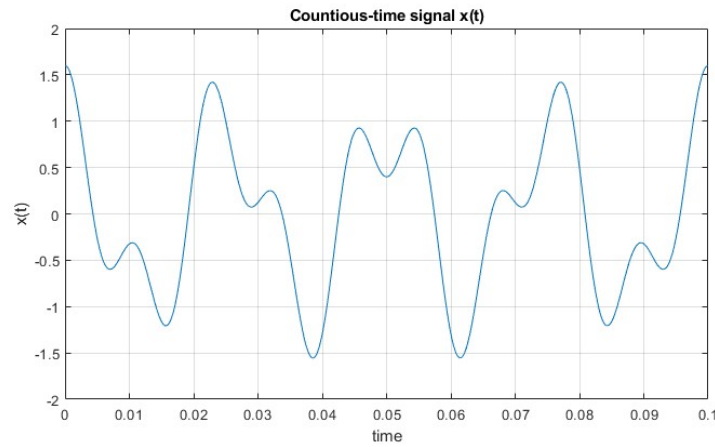


Figure 1: $x(t) = \cos(2\pi \cdot 40t) + 0.6 \cos(2\pi \cdot 90t)$

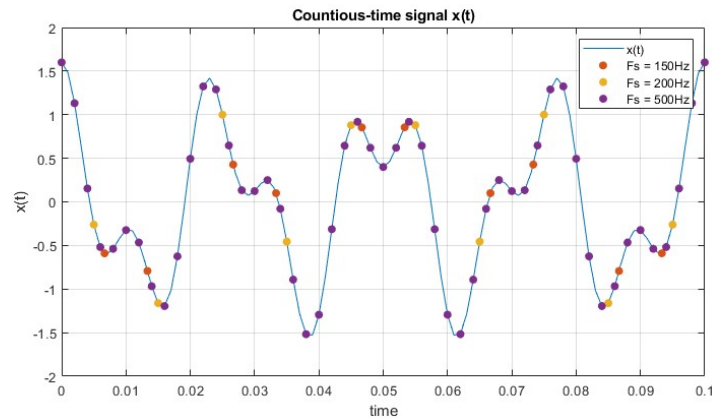


Figure 2: Different sample frequency values are shown along with the original signal.

1.2 Reconstruction

1.2.1 Result

We construct sampled signals using sinc-interpolation and the result is shown in the Figures 3 to 5.

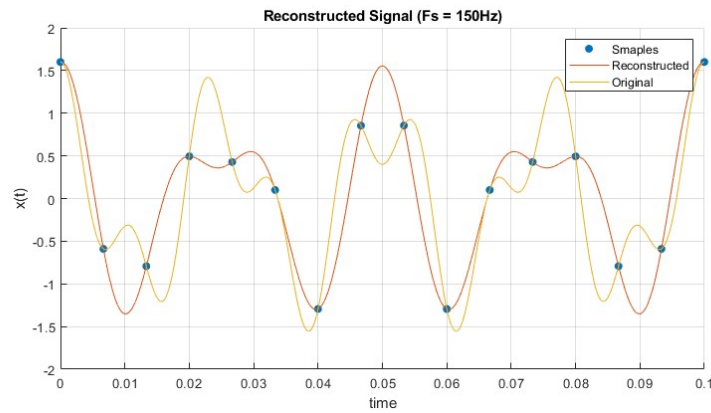


Figure 3: Original and reconstructed signal

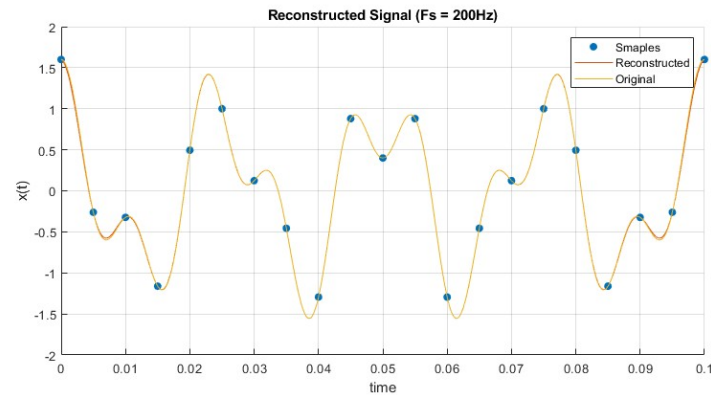


Figure 4: Original and reconstructed signal

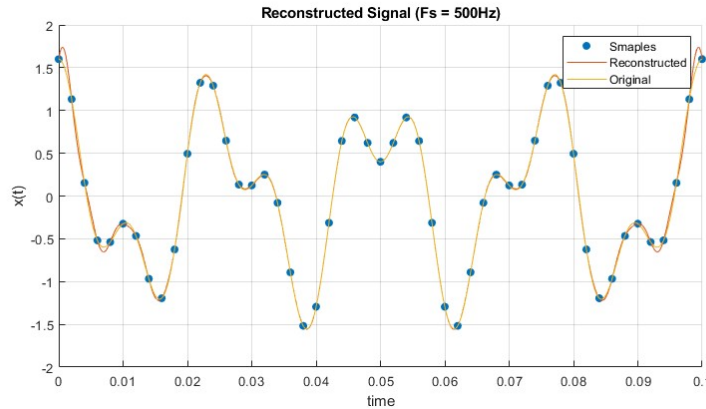


Figure 5: Original and reconstructed signal

We encountered an anomaly. It seems like the 200Hz sampling rate has lower error than 500Hz. This effect is normal, though it feels incredibly counterintuitive. Theoretically both frequencies are able to construct exact signal. By cutting off the signal at 0 and 0.1 seconds, we make artificial sharp edges that troubles sinc-interpolation. The sinc interpolation tries to reconstruct these nonexistent sharp edges, which causes extreme oscillation near the boundaries. If you look at the boundaries you can see this effect.

At 500 Hz, the interpolation has a Nyquist bandwidth of 250 Hz. Because the bandwidth is so wide, the math attempts to accurately trace that artificial sharp drop-off at the edges. This generates high-amplitude, high-frequency (up to 250 Hz) Gibbs ringing that bleeds into your time window.

At 200 Hz, the Nyquist bandwidth is exactly 100 Hz. The interpolation is mathematically physically incapable of generating frequencies above 100 Hz. Because your true signal is 90 Hz, this tight bandwidth perfectly preserves your original signal while acting as a strict low-pass filter that blocks the high-frequency truncation artifacts.

1.2.2 Code

Instead of summation over `n` and `t`, we use the MATLAB built-in matrix multiplication that execute much faster than a simple loop.

```

1 Fs = [150 , 200 , 500];
2 Ts = 1./Fs;
3

```

```

4  t1 = 0:Ts(1):0.1;
5  t2 = 0:Ts(2):0.1;
6  t3 = 0:Ts(3):0.1;
7
8  t = linspace(0,0.1, 50000);
9  x = func(t);
10
11 %Fs = 150Hz
12 [Ts_grid, T_grid] = ndgrid(t , t1);
13 x1 = func(t1);
14 x_r1 = sinc((Ts_grid-T_grid)./Ts(1))* x1(:);

```

2 Aliasing in the Frequency Domain

Increasing frequency when it is still under Nyquist rate, moves one component over frequency domain (the term corresponding to 160Hz cosine.). But above the Nyquist rate, increasing frequency only makes peaks sharper. You can see the effect using the interactive slider in MATLAB.

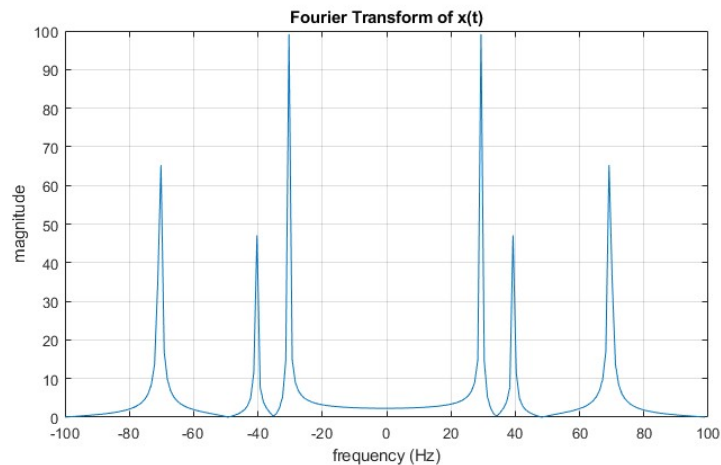


Figure 6: The Fourier transform of the signal with a sampling rate under Nyquist limit.

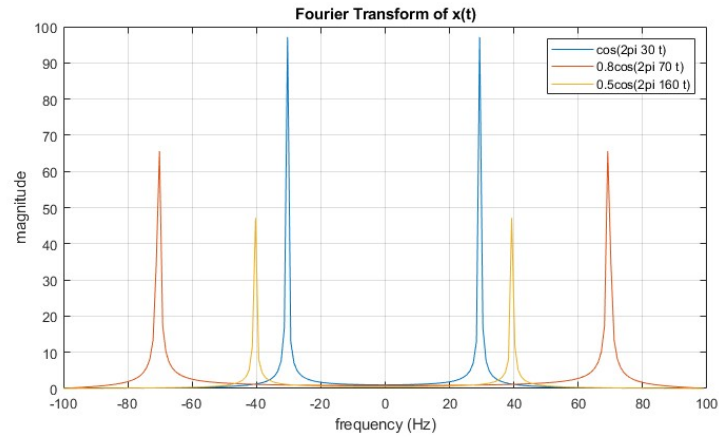


Figure 7: The Fourier transform of the signal with a sampling rate under Nyquist limit. Decomposed to different terms.

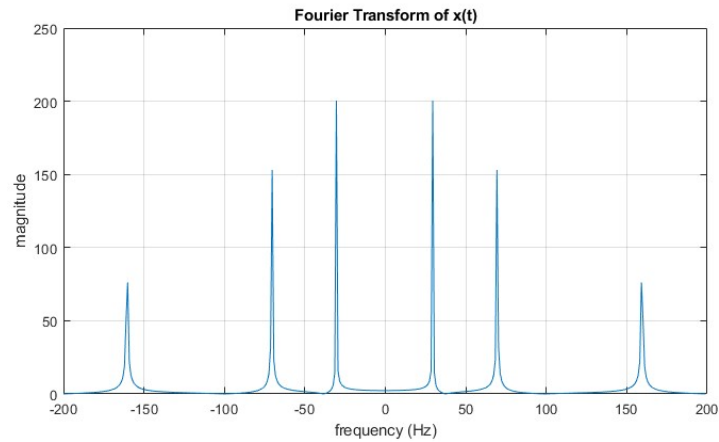


Figure 8: The Fourier transform of the signal which the sampling rate is above Nyquist limit.

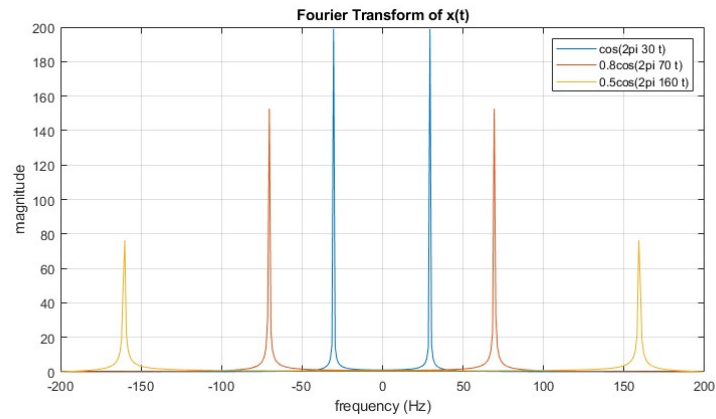


Figure 9: The Fourier transform of the signal which the sampling rate is above Nyquist limit. Decomposed to different terms.

3 Practical Sampling with Zero-Order Hold

3.1 Result

Graphs are shown in Figure 10.

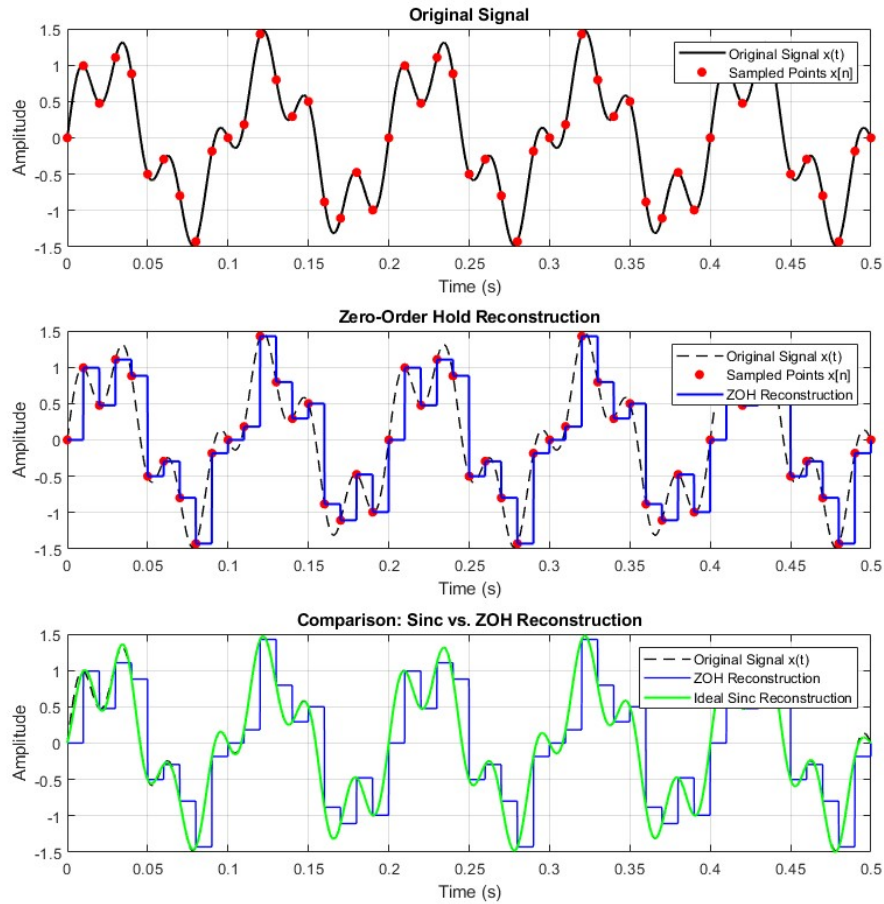


Figure 10

As we expected, error of the ZOH is much higher than ideal sinc interpolation (3 order of magnitude in this case). The ZOH is easier to archive. Simply you need a switch and a capacitor. The capacitor keep the voltage steady then when switch closes again, the voltage level would change to new value.

3.2 Code

We use a built-in MATLAB function called `interp1` for the ZOH reconstruction.

```
1 Fs = 100;
2 Ts = 1/Fs;
3 t = 0:0.0001:0.5;
4 t_n = 0:Ts:0.5;
5
6 x_t = sin(2*pi*10*t) + 0.5*sin(2*pi*35*t);
7 x_n = sin(2*pi*10*t_n) + 0.5*sin(2*pi*35*t_n);
8
9 x_zoh = interp1(t_n, x_n, t, 'previous');
10
11 [Ts_grid, T_grid] = ndgrid(t, t_n);
12 x_sinc = sinc((Ts_grid - T_grid) / Ts) * x_n(:);
13
14 error_zoh = mean((x_t - x_zoh).^2);
15 error_sinc = mean((x_t - x_sinc').^2);
```